

UNIVERSITÀ DEGLI STUDI DI MILANO
Facoltà di Scienze e Tecnologie
Corso di Laurea Magistrale in Informatica

REPRODUCTION AND ANALYSIS
OF REAL-WORLD
USE-AFTER-FREE ATTACKS AND
STUDY OF DEFENCE TECHNIQUES

Relatore: Prof. Andrea LANZI

Correlatore: Dr. Andrea MONZANI

Tesi di:
Stefano NAVA
Matricola: 123456

Anno Accademico 2025-2026

Contents

1	Introduction	1
2	Background	3
2.1	Memory structure	3
2.1.1	Stack	4
2.1.2	Heap	4
2.1.3	Pointers	5
2.1.4	Reference counting mechanisms	5
2.2	Use-after-free (UAF)	6
2.2.1	What causes a UAF?	6
2.2.2	Exploiting UAF and refcount errors	7
2.3	Linux and its kernel	10
2.3.1	The kernel	10
2.3.2	Kernel objects and memory management	11
2.3.3	Use-after-free vulnerabilities in the Linux kernel	12
2.4	Existing tools for detecting UAF vulnerabilities	13
2.4.1	UAFX	14
2.4.2	CountDown	17
3	CVE-2025-21756	23
3.1	Overview	23
3.2	The VSOCK subsystem	23
3.3	Test case presented by the developers	26
3.4	Cause and functions involved	30
4	Setup and tests with UAFX	33
4.1	Setup	33
4.2	Test execution	38
5	Setup and tests with CountDown	39
6	Conclusions	41

List of Figures

1	Structure of memory areas.	4
2	Diagram representing the modular structure of the Linux Kernel.	10
3	Example of code entry points and how they are handled by UAFX.	15
4	Example of refcount-based syscall relation construction.	20
5	Example of a VSOCK communication structure.	24

Chapter 1

Introduction

Software security is nowadays a fundamental aspect of the development and maintenance of IT systems. The increasing complexity of applications, combined with the growing interconnection between devices and services, has significantly expanded the attack surface that can be exploited by potential malicious actors. Operating systems, browsers, drivers and distributed services are critical components whose proper functioning is closely linked to the robustness of their security properties.

In this context, proper memory management plays a central role, particularly in systems developed using low-level languages such as C and C++, which are defined as non-memory-safe. Although these languages offer high performance and control over resources, they are particularly prone to memory management errors, which can result in exploitable vulnerabilities. Memory-safe languages, in fact, have automatic mechanisms for controlling the allocation and deallocation of memory blocks. In the absence of these, while memory access is faster, an important safeguard for security is lost.

Despite decades of research and the development of numerous mitigation techniques, such vulnerabilities continue to arise even in widely used and mature software, highlighting how memory security remains an ongoing challenge in the contemporary cybersecurity landscape.

A particularly relevant example of this challenge is the Linux kernel, one of the most widely deployed and scrutinized pieces of software in existence, yet still subject to memory safety vulnerabilities. Given the complexity of its subsystems and the performance constraints under which it operates, the kernel cannot rely on garbage collection or other automatic memory management strategies. Instead, it largely depends on reference counting, a technique by which each kernel object maintains a counter tracking the number of active references to it, with deallocation occurring only when the counter reaches zero. While reference counting provides a structured and efficient mechanism for controlling object lifetimes, it is not immune to subtle concurrency bugs: incorrect counter manipulations, race conditions on shared objects, or improper ordering of decrement and deallocation operations can all lead to use-after-free (UAF) vulnerabilities, in which a memory region is accessed after it has been freed and potentially reused for a different purpose.

The objective of this thesis is to investigate whether static analysis techniques can reliably detect UAF vulnerabilities arising specifically from reference counting errors in the Linux kernel, a class of bugs that, as will be discussed, has proven difficult to surface using existing automated tools. To this end, we analyze CVE-2025-21756, a real-world vulnerability affecting the vsock subsystem, as a case study: we examine why it eluded prior detection, and what properties of the bug and of the analysis tools contribute to this blind spot.

The thesis is organized as follows:

- **Background:** it covers the fundamentals of memory management: stack and heap organization, pointer semantics and reference counting mechanisms; it introduces the concept of use-after-free vulnerabilities and their relationship with reference counting errors and discusses the Linux kernel architecture with particular attention to how UAF vulnerabilities manifest in that context. The chapter concludes with an overview of existing static analysis tools for UAF detection, with a focused examination of UAFX and CountDown, the two tools that we tested, central to this work;
- **CVE-2025-21756:** after a general overview of the vulnerability, it describes the vsock subsystem and its internal architecture, examines the test case provided by the kernel maintainers, and analyzes the root causes of the bug and the kernel functions involved;
- **Setup and tests with UAFX:** it documents the experimental procedure conducted using UAFX: it covers the environment setup, the compilation of the relevant kernel modules, the extraction of LLVM bitcode and the execution of the analysis, discussing the results obtained and their implications;
- **Setup and tests with CountDown:** TODO

Chapter 2

Background

2.1 Memory structure

Memory is one of the most critical components from a security point of view in a computer system, since during program execution it contains not only the data being processed but also the information necessary to determine the program's behavior. This includes:

- **variables** containing temporary data used by the executed program(s);
- **data structures**, relationships between different types of objects for the purpose of managing more complex data;
- **pointers with memory addresses** which tell the program "where" a certain resource is. They can point:
 - to other objects, to allow the program to read or write data;
 - to functions, to allow the program to execute instructions located at a specific position;
- **information used to manage the execution flow**, such as the address of the first instruction that needs to be executed when returning from a function call.

Unauthorized access to or arbitrary modification of these elements can therefore have consequences ranging from simple data corruption or program crashes to changes in the control flow and, in the most severe cases, the execution of unauthorized operations, which is the subject we're going to examine. To understand the origin and effects of the main memory corruption vulnerabilities, it is therefore useful to first consider how the memory space associated with a program is organized into its main regions.

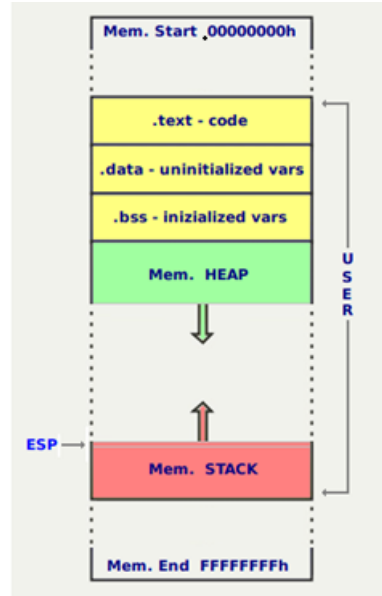


Figure 1: Structure of memory areas.

The two areas primarily affected by attacks of this kind are the stack and the heap.

2.1.1 Stack

The stack is a data structure organised according to a LIFO (Last In, First Out) policy and used primarily for managing function calls. Each function call involves the creation of a *stack frame* containing local variables, parameters and return information (such as the return address, which tells the program the point in the calling code from which to resume execution once the function has been exited). The allocation and deallocation of memory on the stack are automatic and handled by the compiler: when a function terminates, its corresponding frame is removed.

2.1.2 Heap

The heap is a memory region used for dynamic allocations, managed explicitly by the programmer using primitives such as `malloc/free` in C or `new/delete` in C++. These functions do not normally request a new memory region from the operating system for every individual location, but a user-space memory allocator manages larger regions of memory, dividing them into smaller blocks and keeping track of which portions are currently allocated or available for reuse.

On Linux systems, the allocator can obtain additional virtual memory from the kernel through mechanisms such as `brk()`, which changes the end of the process data segment, or `mmap()`, which creates new memory mappings. Large allocations are commonly serviced through `mmap()` while other allocations may be satisfied from memory already managed by the allocator or from an enlarged

process heap. The exact strategy is implementation-dependent and may change according to allocation size and configuration. [1] [2]

Unlike the stack, therefore, heap allocations do not follow a strict LIFO (Last In, First Out) order: allocated blocks may be released independently and subsequently reused for other object, which is essential for dynamically sized and long-lived data structures, but it also makes object lifetime and memory reuse more difficult to manage correctly, creating the conditions in which errors like UAF may appear.

2.1.3 Pointers

In memory-unsafe languages, pointers are identifiers that may contain the address of a specific memory area (to which they "point") and, for example in C, are initialised using the `malloc` function and freed using the `free` function. The programmer can use them to perform operations both on the addresses they contain and on the values contained in the memory cells they point to.

Given their nature, they are one of the main vectors for memory-related attacks. We therefore introduce the concept of a **dangling pointer** [3]: a pointer, if manipulated incorrectly, may continue to point to an area that is no longer valid, typically because it has been deallocated or has expired. This allows an attacker to read or write content to which they should not have access.

Listing 2.1: Example of dangling pointer in C.

```
1 void dangling_pointer_example(void){
2     int *ptr = (int *)malloc(sizeof(int));
3     *ptr = 5;
4     free(ptr); // ptr is now a dangling pointer
5     ptr = NULL; // now it doesn't point to the freed zone anymore
6 }
```

Under the right conditions, exploiting a dangling pointer can lead to **privilege escalation** [4], i.e. a low-level user gaining administrator privileges, thereby putting sensitive data or, in the most serious cases, entire systems at risk.

2.1.4 Reference counting mechanisms

To mitigate the problem of dangling pointers and to avoid placing the responsibility for memory management directly on the programmer, **reference counting** [5] mechanisms have been introduced. These mechanisms associate each object with a counter that tracks the number of active references to it, so that, provided there are no errors, the relevant memory area is freed when it is no longer in use.

Listing 2.2: Simple example of a reference counting mechanism.

```

1 void reference_count_example(void) {
2     int *ptr = malloc(sizeof(int));
3     int ref_count = 1;           // starting with 1 existing reference
4
5     ref_count++;                 // new reference
6     ref_count--;                 // release of a reference
7     ref_count--;                 // release of the last reference
8
9     if (ref_count == 0)
10        free(ptr);
11 }

```

When a new structure gains access to the object, the counter is incremented. Conversely, when the reference is no longer needed, the counter is decremented. When the counter reaches the value *zero*, the system considers the object to be no longer in use and frees up the memory it occupies.

This approach helps to avoid premature deallocations and to coordinate concurrent access to shared resources more reliably. However, errors in counter management (such as multiple decrements or missing increments) can lead to critical conditions.

Early deallocation of an object that is still in use can, in fact, result in *dangling pointers* and use-after-free vulnerabilities, while failure to release references can cause memory leaks and uncontrolled memory consumption. For this reason, reference counting is one of the most delicate mechanisms in memory management within memory-unsafe systems.

2.2 Use-after-free (UAF)

2.2.1 What causes a UAF?

Every system has flaws and is vulnerable to attacks of varying severity. These flaws, which often stem from logical errors or the incorrect management of the lifecycle of objects in memory, can enable an attacker to compromise the integrity of the system, with different objectives. Some of the most known are **buffer overflow** [6] (a process uses a memory area that hasn't been allocated to it) or **double free** [7] (a memory area is deallocated two times corrupting the structures of the memory allocator).

The type of error we're going to focus on is **use-after-free** [8]. Incorrect management of the dynamic memory lifecycle can lead to access to portions of memory that have already been deallocated, allowing an attacker to reuse these areas to inject controlled data and influence the program's behaviour via dangling pointers.

Listing 2.3: Simple example of a UAF exploit in C.

```
1 void uaf_example(void) {  
2     int *ptr = (int *)malloc(sizeof(int));  
3     *ptr = 5;  
4  
5     free(ptr);          // ptr is now a dangling pointer  
6  
7     *ptr = 10;          // use-after-free: access to freed memory  
8  
9     ptr = NULL;         // the pointer is "neutralized"  
10 }
```

A use-after-free vulnerability is fundamentally a violation of the expected lifetime of an object, which passes through three conceptual phases: allocation, usage and deallocation. During the usage phase, one or more parts of the program can hold references to the object and assume that the memory area remains valid. Once the object is deallocated, this isn't true anymore and every reference that could still be used (*dangling pointers*, as explained in section 2.1.3) should be removed to avoid producing use-after-free conditions.

It's important to distinguish the lifetime of an *object* from the one of a *pointer* referring to it. When the `free()` function is called, the allocator marks the memory region as available for future use, but it doesn't remove the existing references to it, which could still be used. In large systems such as the Linux kernel, this distinction becomes particularly relevant because references to the same object may be held by different execution contexts. For example, one thread may remove and free an object from a shared data structure while another thread still retains a pointer to it. If the object's lifetime is not correctly synchronized across these contexts, the remaining reference becomes a dangling pointer and may subsequently lead to a use-after-free.

As mentioned in section 2.1.4, a solution to this problem is counting references for the objects, but an early deallocation can still result in a dangling pointer and can therefore be a specific (but not the only) cause of a use-after-free bug (and a potential exploit as a result).

2.2.2 Exploiting UAF and refcount errors

From an exploit perspective, the dangling reference resulting from this lifetime inconsistency represents only the initial condition of the attack. The object may have already been freed while an execution context still retains its address. The following example illustrates how one can read a value in a freed memory zone exploiting a dangling pointer.

Listing 2.4: Example of a read operation exploiting a UAF caused by a refcount error.

```

1  typedef struct {
2      int value;
3      int refcount;
4  } Object;
5
6  void uaf_refcount(void) {
7      Object *obj = malloc(sizeof(Object));
8      obj->value = 5;
9      obj->refcount = 1;
10
11     Object *ptr = obj;      // new reference
12     /* BUG: missing obj->refcount++ */
13
14     if (--obj->refcount == 0)
15         free(obj);          // obj is freed early
16
17     int value = ptr->value;  // use-after-free
18 }

```

In more complex (and realistic) cases, such as CVE-2021-0920 [9] and CVE-2025-21756 [10], exploitation relies on turning this stale reference into a predictable and controllable interaction with the allocator and with subsequent allocations. In practice, an exploit procedure can be represented by the following sequence of steps:

1. Triggering an incorrect reference-counting transition
2. Maintaining a reachable dangling pointer
3. Reusing the freed memory with a suitable replacement object
4. Accessing the new contents via the obsolete type
5. Transforming the resulting behaviour into a useful (usually malicious) memory primitive

The first stage of an exploit generally involves forcing an incorrect transition in the lifecycle. Depending on the vulnerability, the attacker may need to trigger an error-handling path, repeatedly register and unregister a resource, close an object while it is still in use by another operation, or coordinate multiple concurrent operations. In vulnerabilities relating to reference counting, the key event is often the premature release of the last reference recognised by the system, even though another reachable reference still exists. The attacker must therefore maintain a code path capable of accessing the object after it has been deallocated, while simultaneously causing the counter to reach zero.

This phase can be particularly complex in concurrent systems. One thread might acquire or retain a pointer to an object, while another thread removes the object from a shared structure and frees what the system considers to be the last reference. If the acquisition of the reference is not correctly synchronised with the destruction of the object, we could end having a deallocation between the retrieval of the pointer and the increment of the counter. Similarly, a callback or an asynchronous task may retain an address without actually holding a reference to it. An attacker could attempt to widen these race condition windows by increasing the system's load or using multiple threads.

Once the object has been freed, the dangling pointer alone gives us only limited control. The next objective is to trick the allocator into reusing the freed memory for a new allocation whose contents can be controlled by the attacker (= **heap grooming**, also known as **heap feng shui** or **heap shaping** [11]). The attacker performs a controlled sequence of allocations and deallocations to manipulate the state of the heap and increase the likelihood that a chosen object will occupy the same memory region as the vulnerable object.

The feasibility of this replacement depends heavily on the behaviour of the allocator. Allocators, whether in kernel space or user space, tend to group allocations into size classes and to reuse freed regions for compatible objects. Consequently, the attacker typically seeks a replacement object belonging to the same allocation class as the vulnerable structure and which can be created repeatedly via an accessible interface.

Following memory reuse, two logically distinct objects occupy the same region at different times. The vulnerable code still interprets the dangling pointer according to the layout and semantics of the original object (as it should), while the allocator treats the region as belonging to the new object. This creates a form of type confusion induced by memory reuse: the fields of the new object may be interpreted as flags, sizes, reference counters, pointers to lists or function pointers belonging to the original structure.

The resulting access can provide various exploit primitives. A read via the dangling pointer may reveal heap addresses, kernel pointers or other sensitive values, making it easier to bypass mechanisms such as ASLR (Address Space Layout Randomization [12]: a technique for randomizing addresses of stack, heap and libraries in memory which usually helps preventing memory attacks). If a corrupted field is subsequently used as a pointer or index, we can obtain out-of-bounds read or arbitrary memory access primitives. A write via the obsolete reference, on the other hand, can alter the state of the new object, resulting in a controlled or partially controlled write to another area of memory.

Objects that contain pointers to functions or references to related structures are of particular interest to us, as their corruption can indirectly affect the control flow. However, modern exploits do not necessarily aim to immediately overwrite a function pointer. Mechanisms such as Control-Flow Integrity (CFI) [13], memory non-executability and restrictions on code execution in kernel space often make direct redirection difficult. Instead, the exploit uses the initial vulnerability to construct more powerful read and write primitives, obtain address information and modify sensitive data structures. These primitives can then be exploited to

alter credentials, permissions or the state of a process, depending on the execution context.

A pending reference may also interact directly with the reference counting mechanism. If the obsolete pointer is used in a deallocation operation, the system may decrement a field that no longer represents the original counter. This may corrupt the new object or cause a second deallocation of the same memory region. In such cases, the initial error may lead to additional conditions such as double-free or premature destruction of the new object. The exploit may therefore involve multiple consecutive violations of the object's lifecycle.

Although this is only a generalized explanation of the idea and process behind exploiting this type of vulnerability, it usually isn't easy as it seems, due to the need for certain conditions to be met.

2.3 Linux and its kernel

2.3.1 The kernel

The kernel is the central component of a Linux operating system and operates with higher privileges than normal programs running in user space. Its main task is to mediate access to hardware resources and to provide applications with a controlled set of services via system calls. This is convenient for the developer, but it also carries some risks: while a bug in a user space program is confined to its process, memory corruption in kernel space can compromise the stability of the entire system or allow operations to be carried out with elevated privileges (the *privilege escalation* that we mentioned in section 2.1.3).

In addition to this, Linux keeps a highly modular structure, as some functionality can be compiled as a loadable module.

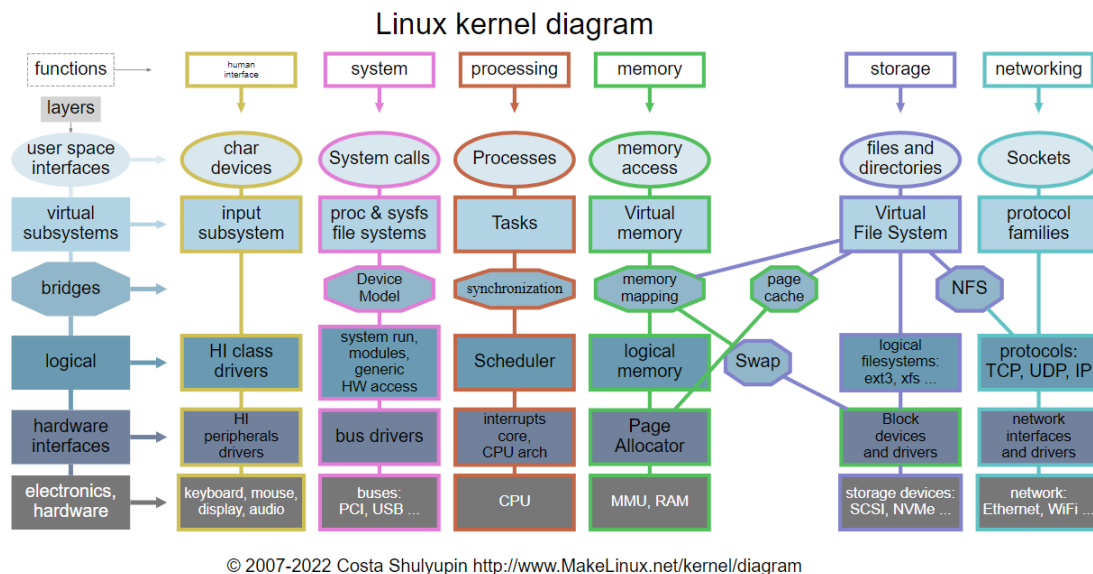


Figure 2: Diagram representing the modular structure of the Linux Kernel.

However, this flexibility does not provide complete isolation: a loaded module

continues to run in kernel space and can access the kernel’s internal structures, with the same aforementioned security implications.

Since its first release in 1991, Linux has evolved from a small-scale project into a platform used on a vast number of servers and devices all around the world, as we can see on the following table which sums up the distribution of Linux-powered machines around the world [14]:

Metric	Linux Share
Server OS Market (2026 est.)	51.3%
Websites (known OS)	61.3%
Top 1 Million Web Servers	96.3%
Government Data Centers	64.9%
Top500 Supercomputers	100%

This evolution has been accompanied by the introduction of numerous hardware architectures, new drivers and subsystems, which support a broader quantity of features but bring more security and stability issues on the kernel’s side. The interfaces exposed to user space are kept as stable as possible, while the kernel’s internal APIs may change significantly between different versions.

2.3.2 Kernel objects and memory management

Unlike operating systems such as Windows, where the concept of *kernel object* is formally defined and exposed through a unified object manager [15], the Linux kernel does not adopt a single, centralized abstraction for its internal entities but takes advantage of a rich set of well-defined C structures that represent the fundamental components of the system (e.g. processes, files, network sockets, memory regions). Each of these structures encapsulates both the data describing the entity and the state required to manage its lifecycle.

A representative example is `task_struct` [16]: this structure is used by the kernel to represent a process or thread. It contains hundreds of fields: scheduling information, memory maps, open file descriptors, credentials, and more; it is one of the most referenced structures throughout the kernel codebase. Similar in spirit are structures such as `struct socket` [17] and `struct sock` [18] in the networking subsystem, `struct inode` [19] for filesystem objects, or `struct sk_buff` [20] for network packets.

These objects are dynamically allocated on the kernel heap through the *slab allocator* [21] (in its modern SLUB implementation [22]), a memory management subsystem optimized for the frequent allocation and deallocation of fixed-size objects. The slab allocator maintains per-type caches of pre-allocated memory, reducing fragmentation and allocation overhead but also, as will be discussed, creating conditions that are relevant to the exploitation of memory corruption vulnerabilities.

Since many of these objects can be referenced simultaneously from multiple kernel subsystems (e.g. a socket can be held by a process, referenced by a pending timer, and pointed to by an active network connection at the same time) their lifetime cannot be tied to a single owner and this is where the reference counting mechanism comes into play. It is efficient, but its correctness relies entirely on every code path that acquires or releases a reference doing so properly. As the kernel grows in complexity and concurrency, maintaining this invariant across all subsystems becomes increasingly difficult and subtle errors in reference counter management can cause the counter to reach zero prematurely, triggering the deallocation of an object that is still reachable from some other part of the kernel.

2.3.3 Use-after-free vulnerabilities in the Linux kernel

Use-after-free vulnerabilities are particularly relevant in the Linux kernel because many kernel objects are dynamically allocated, shared among different subsystems and accessed concurrently by multiple execution contexts (e.g. sockets, network objects, device structures). Their lifetime, as described in section 2.1.4, is often managed through mechanisms like reference counting which, in case of wrong manipulation, can produce dangling pointers and subsequent use-after-free exploits.

As stated previously, the consequences of errors like these are generally more severe than in a normal user-space program, since kernel code executes with high privileges. Detecting these vulnerabilities is complicated by the structure of the kernel itself. The allocation, use and release of an object may occur in different context and the relevant sequence of operation is likely not visible within a single control-flow path (this includes the thread concurrency problem seen in section 2.2.2).

These characteristics make automated kernel analysis particularly complex. Detection tools have to deal with code spread across multiple modules that interact in a non-linear manner. Their analysis may also be constrained to a specific kernel version or compiler, or require modifications to the build system and, in some cases, full kernel instrumentation. Moreover, the continuous evolution of internal APIs can quickly lead to compatibility issues if the analyzer is not actively maintained. The kernel documentation distinguishes between numerous categories of tools, such as:

- *static analyzers*: code analysis by tracking objects' paths without executing the code. An example is UAFX [23], which we tested and discuss in section 2.4.1
- *sanitizers*: suspect behaviour detection by instrumentating the code through a compiler
- *code coverage tools*: measurement of the quantity of code covered by tests
- *fuzzing tools*: generation of random inputs in order to test different execution paths for the analyzed software

- *mechanisms for detecting race conditions*: identification of critical code sections where a race condition may happen

LLVM

Most of the tools we have considered are based on LLVM [24], a modular infrastructure for building compilers and code analysis tools. Its central component is an intermediate representation (generated by a compiler; one of the most used is Clang [25]), commonly referred to as LLVM IR, on which modifications can be made that are independent of the source language and, to some extent, of the target architecture.

The use of LLVM is significant in the field of security because it allows analyses and transformations to be incorporated directly into the compilation process (e.g. by adding checks that will be executed at runtime). This integration enables more in-depth analyses than simply processing the source code textually, albeit at the expense of the performance of the system being analysed; however, a system compiled with LLVM will likely not be made available to the end user but will be used solely for analysis purposes.

2.4 Existing tools for detecting UAF vulnerabilities

For all known bugs, there is a wide range of tools described in the academic literature, used either individually or in combination, that serve (or help) to detect them. UAF issues, however, especially when related to reference counting problems, are particularly challenging. Due to the unpredictable nature of these bugs, the numerous conditions required to exploit them, and the variety of contexts in which they can occur, identifying them is by no means easy. In addition to the identification process itself, other problems may arise, such as large volumes of false positives that can cause a tool to be inaccurate on its own and require human review to distinguish them from true positives, or the need for extreme computational power to run the tool effectively.

Below are some examples of tools capable of analyzing code and detecting use-after-free vulnerabilities. Since each vulnerability exploits different weaknesses, we have compiled a list of tools with different purposes and modes of operation to provide a broader range of options in this regard.

Tool	LLVM//based	Usable in kernel	Public source	Supports refcount
UAFX [23]	Yes	Yes	Yes	No
CountDown [26]	Yes	Yes	Yes	Yes
Infer [28]	Yes (+ Clang)	Yes	Yes	No
Clang Static Analyzer [25]	Yes	Yes	Yes	No
Syzkaller [27]	Yes	Yes	Yes	No
GCC-fanalyzer [29]	No	No	Yes	No
CRed [30]	Yes	No	No	No
Cppcheck [31]	Yes	No	Yes	No
SVF-derived tools [32]	Yes	No	Yes	No
FreeWill [33]	Yes	No	No	Yes
LinkRID [34]	Yes	Yes	No	Yes
CRCount [35]	Yes	No	No	Yes
UAFChecker [36]	Yes	No	No	No
DCUAF [37]	Yes	Yes	No	No
SinkFinder [38]	Yes	No	No	Yes
Goshawk [39]	Yes	Yes	Yes	No
CID [40]	Yes	Yes	No	Yes

In the next sections, we will describe the tools that are relevant for this work. There were two tools specifically designed to detect UAF bugs in the Linux kernel: CountDown and CID. Unfortunately, CID does not have publicly available source code, and, as with other tools that do not meet this criterion, we decided to exclude them from the selection.

Since our focus was on a use-after-free vulnerability involving a reference-counting issue, we also excluded some tools whose papers clearly stated that they would not be effective for this purpose.

Finally, we excluded the tools that would not have worked on the kernel because they were designed for smaller, less complex sections of code.

After considering these factors, the decision was made to choose UAFX and CountDown.

2.4.1 UAFX

UAFX is a static analyzer, presented at NDSS 2025 [23], designed to detect *cross-entry* (non-linear) use-after-free vulnerabilities in the Linux kernel. The term "cross-entry" refers to conditions in which the deallocation and subsequent use of an object occur through different entry points (e.g system calls, callbacks,

driver interfaces). In such cases, the link between the deallocation point and the usage point may involve parts distributed across distinct execution paths, making it difficult to detect using analyzers limited to a single function or a single call chain (i.e. where memory allocation, usage, and deallocation occur in a linear pattern). UAFX’s approach is based on achieving two main macro-objectives:

- **Escape-Fetch-based Cross-Entry Alias Analysis**
- **Systematic Partial-Order Constraint-based UAF Validation**

We describe them with reference to the following figure:

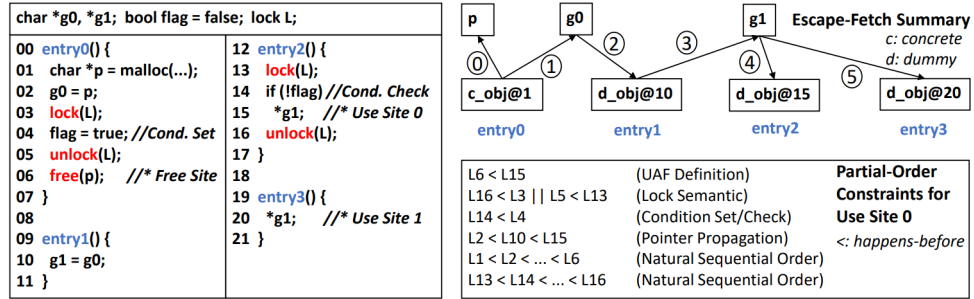


Figure 3: Example of code entry points and how they are handled by UAFX.

Escape-Fetch-based Cross-Entry Alias Analysis

UAFX treats object assignments to pointers as *escapes* and retrievals (from “unknown” areas) as *fetches*. UAFX attempts to construct Escape-Fetch Paths (EFPs) to define aliases that track memory allocation, usage, and deallocation from different access points. This is necessary because, from the perspective of a single function, an object may have an unknown origin (as is commonly the case with an externally initialized pointer, such as `g1` in `entry3`).

Systematic Partial-Order Constraint-based UAF Validation

Once the aliases/paths have been confirmed, UAFX verifies that the conditions for a UAF actually exist. If, for example, there is a check on a flag (`entry0`, `entry2`) that is manipulated by multiple entry functions and used to prevent a UAF on a critical pointer, that path must be discarded.

Workflow

We can break down UAFX’s workflow as follows:

1. **Identifying entry functions.** This is done semi-automatically, starting with:
 - *General call analysis:* functions that serve as entry points typically have no callers in the program, so they are candidates.

- *Software-based heuristics*: the paper uses drivers as an example; these typically have entry points in a `file_operations` structure, which makes them easy to identify.

After these initial automated steps, a manual review is conducted to exclude unnecessary candidates or to add others.

2. **Code analysis and summary for each entry.** The entry functions are analyzed, and all escape/fetch objects, critical regions, and condition sets/checks (such as the flag in the example) are identified. UAFX generates a summary of all possible paths (in Figure 3, the graph in the upper right).
3. **Cross-entry detection of free and use sites with aliases.** UAFX considers all sites where memory regions are freed and used as potential candidates for UAF. In the example, with the path `entry0-entry3`, defined as follows (between the *free* operation on line 6 and the *use* operation on line 20):

$$(c_obj@1 \longrightarrow g0 \longrightarrow d_obj@10 \longrightarrow g1 \longrightarrow d_obj@20)$$

there is a potential UAF.

4. **UAF Validation.** UAFX considers all candidates and evaluates their constraints within the path (order, locks, set/check conditions) to determine whether each one is actually a vulnerability. It also verifies that the free and use sites actually access the same object, taking into account the entire escape-fetch path. For this purpose, SMT solvers can be used (the paper cites z3 [43] as an example).

Thread tracking

Since some UAFs are caused by race conditions, UAFX recognizes two types of thread-related lifecycles:

- **Thread creation** (`pthread_create`, `fork`): calls that create child threads. UAFX retrieves information from these events (e.g., the thread’s entry function or parameters)
- **Thread joins** (`pthread_join`, `wait`): events that ensure threads exit before a certain point in the parent thread. UAFX pairs each exit event with its corresponding creation event to track the entire thread lifecycle.

UAFX limitations and extensions

Although UAFX is designed to automatically identify use-after-free vulnerabilities, its analysis deliberately excludes some classes of candidates in order to preserve high precision and limit the number of false positives.

A first limitation concerns the identification of entry functions. As described previously, UAFX automatically considers functions with no callers as potential

entry points, but relevant functions may themselves be invoked by other functions and therefore not satisfy this criterion. UAFX addresses this limitation by allowing additional entry functions to be manually included in the candidate set during the second stage of the identification process.

A more significant limitation concerns reference-counting mechanisms. As explicitly discussed by the authors in the *Reference Count Mechanisms* section of the UAFX paper [23], candidates involving reference counting are intentionally excluded from the analysis. Tracking the lifetime of reference-counted objects introduces additional complexity and may produce a large number of false positives. The authors therefore favor higher precision at the cost of reduced recall for this class of lifetime-management errors.

This limitation does not imply that reference-count-related vulnerabilities cannot be addressed through static analysis. Other tools focus specifically on the analysis of reference-counting mechanisms and may complement the approach adopted by UAFX. In particular, LinKRID [34], CRCCount [35], and FreeWill [33] are discussed in relation to the identification of reference-counting errors. Their analyses could potentially be combined with or used to extend UAFX, although doing so would require adapting the respective analyses and reconciling their different representations of object lifetime and reference-management operations.

Such an extension would broaden the class of lifetime errors considered by UAFX, potentially improving its ability to detect use-after-free conditions originating from incorrect reference-count management. However, this would also introduce the precision-recall trade-off that UAFX deliberately avoids in its original design.

2.4.2 CountDown

CountDown is a fuzzer for the Linux kernel, presented at CCS 2024 [26] and, unlike UAFX, it's specifically designed to detect memory timing errors caused by incorrect handling of reference counters. Unlike traditional fuzzers, which are primarily driven by code coverage, CountDown uses operations on reference counters as feedback to identify sequences of system calls that could potentially alter the lifecycle of those same objects. The tool is built on top of Syzkaller [27] (an open source, unsupervised, coverage-guided kernel fuzzer developed by Google) and extends its strategies for generating and mutating test programs.

During execution, CountDown instruments the kernel to associate increments, decrements, and accesses to reference counters with the system calls that triggered them. Based on this information, it establishes relationships between system calls that operate on the same counters and assigns them a higher probability of being combined in the generated tests. The tool can also repeatedly insert operations that decrement a counter, in an attempt to trigger the object's deallocation, followed by operations that continue to access it. In this way, fuzzing is directed toward sequences that can more effectively expose UAFs caused by incorrect operations on counters.

CountDown's approach is based on the idea of refcount-driven fuzzing to achieve two macro-objectives:

- *Identification of refcount-based relationships between syscalls*: instead of relying solely on syscall signatures or coverage, CountDown correlates syscalls that operate on the same object (*shared refcounts*).
- *Systematic exposure of UAF bugs*: through targeted mutations, the tool attempts to force an object's refcount to zero and subsequently access that object via syscalls that share its reference.

Workflow

CountDown's workflow can be broken down into three main phases:

1. **Kernel instrumentation and logging**: CountDown instruments the refcount APIs in the Linux kernel to log every refcount operation using a 4-element tuple:

$$\langle S, O, \delta, V \rangle$$

where a syscall S updates the refcount O by a value δ . V is the final value.

The instrumented APIs are listed in the following table:

Name	Description
<code>refcount_set</code>	set refcount to the given value
<code>refcount_add</code>	add the given value to refcount
<code>refcount_inc</code>	increase refcount by 1
<code>refcount_dec</code>	decrease refcount by 1
<code>refcount_add_not_zero</code>	add given value to refcount if refcount is not 0
<code>refcount_inc_not_zero</code>	increment if the refcount is not 0
<code>refcount_dec_not_one</code>	decrement if the refcount is not 1
<code>refcount_dec_if_one</code>	decrement if the refcount is 1
<code>refcount_dec_and_test</code>	decrement refcount and check if result is 0
<code>refcount_sub_and_test</code>	subtract value from refcount and check it with 0

To identify the same object across different virtual machines (VMs), it uses a checksum of the call stack at the time of the object's initialization through `refcount_set`.

CountDown creates a RAI map (`refcount-address-ID`) to link the addresses of the refcounts with their refcount IDs.

When a refcount is initialized, a new entry in the RAI map is created with

its address and ID.

When a refcount reaches zero, the entry is removed from the map.

When a refcount is incremented or decremented, CountDown searches the address in the map to find its ID and records the operation.

During fuzzing, to connect refcount operations with syscalls, CountDown needs to record the Syzkaller syscall number (NR) for each operation. This is done in a thread-safe manner in order to avoid race condition, since the Linux kernel utilizes multiple threads to execute parallelized syscalls.

While tracking the refcount values, CountDown considers that one syscall may modify a refcount many times, but it only cares about the final variation (δ). In order to do so, every refcount API listed in table 1 is modified to record every single refcount change. This data is kept in a two-dimension table (one for the syscall number, one for the refcount ID), in which every entry records the current δ and the current V .

2. **Reworking relationships between system calls:** CountDown constructs multiple relations among syscalls and refcounts starting from the refcount operations mapped in the 4-tuple format (S, O, δ, V) .

First, it identifies syscalls that effectively change the refcount of any object. It remaps a set of 4-tuples in a new map, where the key is the refcount ID and the value is a set of syscalls and their changes to the refcount (δ), avoiding relations between syscalls from different programs (which can quickly become unreliable).

Based on the shared refcounts, every relation is reworked in a new one defined as **RefcntRelation**. The assumption is that, if two syscalls access the same refcount within one execution, they have a stronger relation with respect to triggering more complicated refcount operations. Each syscall set associated with a specific refcount is then split into a set of syscalls pairs. Following that, CountDown creates the syscall relation table, where each unique pair of syscalls has an entry and the value is the number of shared refcounts accessed by both.

As explained in the paper, this newly constructed relation is more efficient in detecting refcount-related UAF bugs. However, the kernel code base is very large and refcount operations are not densely distributed. To solve the problem, the **RefCntRelation** is merged with the Syzkaller’s standard **SyzRelation** and handled based on the fuzzing time. At the beginning of fuzzing, CountDown assigns a high priority to the original coverage-based relation to quickly explore more kernel code. As time goes on, it allocates more weight to CountDown’s own relation so that the fuzzer can focus on testing complex refcount operations. This can be represented as the following formula:

$$OverallRelation = \log_2(SyzRelation) + k * \log_2(RefcntRelation)$$

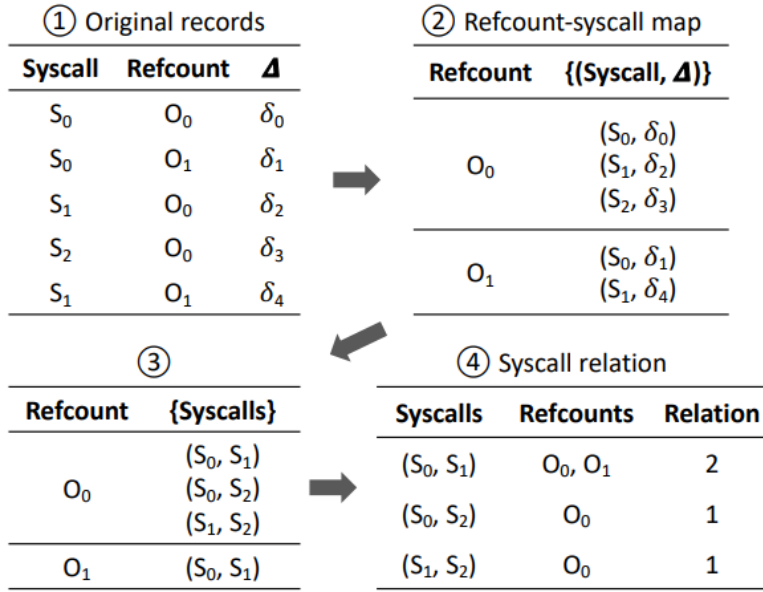


Figure 4: Example of refcount-based syscall relation construction.

k starts with a small value and is incremented every time after testing a fixed number of test cases.

Since fuzzers are non-deterministic, different kernel instances can gather different relations. When a new one is received from one VM, CountDown synchronizes it across all instances (to ensure that the other instances can directly make use of it) by fetching each VM’s report, merging them and then distributing the result to all instances.

3. **Guided program mutation:** CountDown uses the learned relationships to generate new programs. It starts by using the mutators from previous fuzzing tools (like Syzkaller) to mutate the program. Having assigned higher weights to refcount-based relations, the mutation will prefer combining syscalls that are strongly related regarding refcount operations.

Then it creates a new mutator that combines refcount-related syscalls more aggressively. CountDown chooses one target program and runs it in order to collect all accessed refcounts. It will update the refcount-syscall map if the execution triggers new records. To mutate the program, it will randomly select a refcount from the collected set. If a refcount was accessed multiple times, it will have a higher probability to be selected.

CountDown then searches the map for syscalls that operate on this refcount and selects one for insertion into the program, which is done reusing Syzkaller’s logic, preferring a later position, because the syscalls at the beginning may work on the initialization of resources and refcounts.

In addition to new code coverage, CountDown considers any program that exhibits a new `(syscall, refcount)` pair never seen before to be “interesting” and saves it for future changes. This is done while fuzzing through

checking whether the execution triggers new code coverage or a new refcount operation, which was previously missed. If that's the case, it saves the new program into the input queue for further mutations and updates the refcount-syscall map with the new entry.

As a final but important point, a triggered refcount error will not immediately result in a memory error. CountDown needs to trigger more operations to make the kernel release the object and access it through dangling references. It does so by inserting more refcount-related syscalls into the executed program in order to reduce the gap between a refcount bug and a UAF, so that sanitizers like KASAN can capture it.

Limitations

Although CountDown is optimized to detect UAFs with refcounts, it still has some limitations:

- **Generic refcounts:** some refcounts (such as those in the AppArmor security module) are used by nearly all syscalls, creating "noise" and artificial relationships that CountDown must filter out to maintain accuracy.
- **Coverage trade-off:** because it focuses heavily on refcount logic, CountDown may record slightly lower total code coverage (approximately 1.8% – 4.3% less) than Syzkaller, as it ignores paths unrelated to object management.
- **Instrumentation overhead:** although optimized through selective logging (excluding interrupts and asynchronous tasks), the constant monitoring of every refcount operation introduces a higher computational load than traditional fuzzing.

Chapter 3

CVE-2025-21756

3.1 Overview

CVE-2025-21756 is a use-after-free vulnerability identified in the Linux kernel's AF_VSOCK subsystem, which is used for communication between hosts and virtual machines. The vulnerability allows a local attacker to gain elevated privileges (up to root) by exploiting incorrect management of the socket object lifecycle within the kernel.

The problem arises during specific sequences of operations on vsock sockets, leading to the dereferencing of memory structures that have already been freed and, consequently, to kernel memory corruption. Specifically, an attacker could exploit this vulnerability to achieve privilege escalation.

3.2 The VSOCK subsystem

VSOCK is a component of the Linux kernel that manages communication between a virtual machine (*guest*) and the system hosting it (*host*). It was developed in response to the need to provide a method of communication between the two entities that offers better performance than traditional network interfaces. Unlike TCP/IP sockets, which are based on IP addresses and network protocols, vsock sockets use an addressing model specific to virtualised environments. Communication via vsock utilises a pair of identifiers:

- **CID (Context IDentifier)**: uniquely identifies an endpoint (host or VM)
- **Port**: similar to TCP/UDP ports

Some relevant CID values may include:

- VMADDR_CID_HOST: represents the host
- VMADDR_CID_LOCAL: local communication
- specific CIDs assigned to individual VMs

The advantage of this model is that it avoids the overhead associated with the IP stack.

Communication takes place via two transport networks:

- **G2H (guest-to-host)**: these are executed on the guest and are usually device drivers. We distinguish between three types:
 - *virtio-transport*: typical of KVM/QEMU, it uses virtqueues located within the guest as buffers, but the host also reads from and writes to them.
 - *vmci-transport*: VMWare’s proprietary for H2G/G2H.
 - *hyperv-transport*: based on Hyper-V VMBus, designed specifically for Microsoft environments.
- **H2G (host-to-guest)**: these are run on the host and usually provide device emulation for the guest. There are two types:
 - *vhost-transport*: host-side backend for virtio-transport.
 - *vmci-transport*: VMWare’s proprietary for H2G/G2H.

To illustrate communication between the host and the guest, we can use QEMU, an open-source emulator and virtualisation program, as an example. When creating a virtual machine, the structure will be as follows:

- *virtio-transport* on the guest
- *vhost-transport* on the host: provides in-kernel device emulation to the guest and uses `ioeventfd` and `irqfd` to receive and send events to the guest.

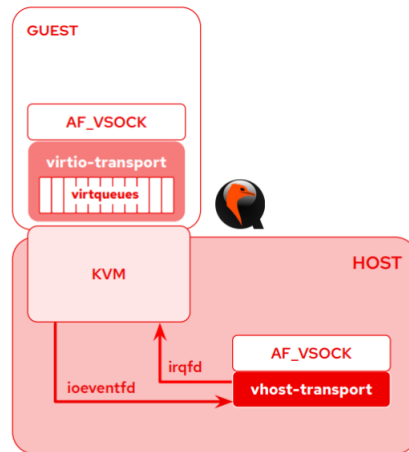


Figure 5: Example of a VSOCK communication structure.

Both provide an interface to the socket layer via the vsock core and exchange packets with virtio queues (*virtqueues*).

KVM (Kernel-based Virtual Machine) turns the host OS into the VM’s hypervisor, enabling it to manage the CPU and memory directly.

Virtqueues

These are shared buffer queues in memory that allow the two parties to communicate bidirectionally (even though a single virtqueue is dedicated to a single direction) while maintaining low overhead.

In **split virtqueue** mode (classic), they are based on three main shared memory areas:

- **Descriptor Area:** contains the buffer metadata (the guest's physical address and length).
- **Available Ring:** a list of buffers that the driver has added and which are ready to be consumed by the host device.
- **Used Ring:** a list of buffers that the device has finished processing and is returning to the driver.

The **packed virtqueue** mode, on the other hand, uses a more modern and compact layout, merging the available and used rings, thereby improving the performance of the processor cache.

The operation can be broken down into the following stages:

1. **Insertion:** the VirtIO (guest) driver places the buffers in the *descriptor area* and updates the *available ring*.
2. **Notification (Kick):** The driver sends a notification to the device ("doorbell") to indicate that new data is available.
3. **Processing** of data by the host.
4. **Completion:** the host updates the *used ring* and sends an interrupt to the guest to signal completion.

The virtqueue backend (which actually manages the queue on the host) can be **QEMU**, with full emulation, as in the case under consideration, or **vhost**: the backend runs within the host kernel (vhost-net/vhost-blk), allowing the driver to communicate directly with the kernel without going through QEMU's user space, thereby drastically reducing latency.

Internal architecture of VSOCK

From an implementation perspective, each vsock socket is represented in the kernel by a data structure (`struct vsock_sock`), which extends the generic socket structures (`struct sock`). These objects are managed via:

- **status lists:**
 - *bound sockets*: associated with a port;
 - *unbound sockets*: created but not yet associated;
- **connection queue:** handling of incoming requests;

- **reference counting**: a fundamental mechanism for managing the life cycle of objects.

When consistency in the management of these elements is lacking, there is a risk of encountering the problems mentioned above, exposing the system to risks and attacks.

The life cycle of a VSOCK socket

Typically, a vsock socket goes through the following stages:

1. **Creation** (`socket()`): memory is allocated for the data structure, which is then added to the *unbound* list.
2. **Binding** (`bind()` or `autobind`): the socket is associated with a port and is consequently added to the *bound* list.
3. **Connection** (`connect()` / `accept()`): the actual communication channel is established.
4. **Release** (`close()`): when the socket is no longer needed, it is removed from the internal structures, its reference count is decremented and, if this reaches zero, its memory is deallocated.

3.3 Test case presented by the developers

Let's start with a test case provided by the maintainers of the VSOCK module for the Linux kernel, who, after releasing the patch, made it available to allow users to check whether their system is affected by the issue.

Listing 3.1: Test case presented by the developers.

```

1  #define MAX_PORT_RETRIES      24
2  #define VMADDR_CID_NONEXISTING 42
3
4  /* Test attempts to trigger a transport release for an unbound
5   * socket. This can lead to a reference count mishandling.
6   */
7  static void test_seqpacket_transport_uaf_client(
8      const struct test_opts *opts
9  ) {
10     int sockets[MAX_PORT_RETRIES];
11     struct sockaddr_vm addr;
12     int s, i, alen;
13
14     s = vsock_bind(
15         VMADDR_CID_LOCAL,
16         VMADDR_PORT_ANY,
17         SOCK_SEQPACKET
18     );
19
20     alen = sizeof(addr);
21     if (getsockname(s, (struct sockaddr *)&addr, &alen)) {
22         perror("getsockname");
23         exit(EXIT_FAILURE);
24     }
25
26     for (i = 0; i < MAX_PORT_RETRIES; ++i) {
27         sockets[i] = vsock_bind(
28             VMADDR_CID_ANY,
29             ++addr.svm_port,
30             SOCK_SEQPACKET
31         );
32     }
33
34     close(s);
35     s = socket(AF_VSOCK, SOCK_SEQPACKET, 0);
36     if (s < 0) {
37         perror("socket");
38         exit(EXIT_FAILURE);
39     }
40
41     if (!connect(s, (struct sockaddr *)&addr, alen)) {
42         fprintf(stderr, "Unexpected connect() #1 success\n");
43         exit(EXIT_FAILURE);
44     }
45     /* connect() #1 failed: transport set, sk in unbound list. */
46
47     addr.svm_cid = VMADDR_CID_NONEXISTING;

```



```

48     if (!connect(s, (struct sockaddr *)&addr, alen)) {
49         fprintf(stderr, "Unexpected connect() #2 success\n");
50         exit(EXIT_FAILURE);
51     }
52     /* connect() #2 failed: transport unset, sk ref dropped? */
53
54     addr.svm_cid = VMADDR_CID_LOCAL;
55     addr.svm_port = VMADDR_PORT_ANY;
56
57     /* Vulnerable system may crash now. */
58     bind(s, (struct sockaddr *)&addr, alen);
59
60     close(s);
61     while (i--)
62         close(sockets[i]);
63
64     control_writeln("DONE");
65 }

```

The test case in question does not lead to a privilege escalation, but causes the kernel to crash on vulnerable systems. By examining it step by step, it is possible to understand what is going wrong in the reference counting mechanism associated with a vsock socket.

Step-by-step analysis

Let's look at every relevant step in the test case. Every involved function will be described in detail in section 3.4.

```

14     s = vsock_bind(
15         VMADDR_CID_LOCAL,
16         VMADDR_PORT_ANY,
17         SOCK_SEQPACKET
18     );

```

After initially declaring the variables, a new local (identified by CID = 1) socket is bound. By doing this, the integer variable `s` will contain a new valid port assigned from the kernel.

```

21     if (getsockname(s, (struct sockaddr *)&addr, &alen)) {
22         perror("getsockname");
23         exit(EXIT_FAILURE);
24     }

```

The instruction inside the `if` condition puts into `addr` the address of the new-bound socket. It terminates with a failure if `getsockname` returns `-1` (negative result) while it continues if it returns `0` (positive result).

```

26     for (i = 0; i < MAX_PORT_RETRIES; ++i) {
27         sockets[i] = vsock_bind(
28             VMADDR_CID_ANY,
29             ++addr.svm_port,
30             SOCK_SEQPACKET
31         );
32     }

```

The `for` loop arbitrarily binds 24 ports (unlike the instruction in row 14, where it is automatically assigned by the kernel), starting with the port immediately following the one assigned by the kernel, in order to occupy them.

```

34     close(s);

```

The first socket is then closed. Its only purpose was to automatically obtain the first port from the kernel.

```

35     s = socket(AF_VSOCK, SOCK_SEQPACKET, 0);
36     if (s < 0) {
37         perror("socket");
38         exit(EXIT_FAILURE);
39     }
40
41     if (!connect(s, (struct sockaddr *)&addr, alen)) {
42         fprintf(stderr, "Unexpected connect() #1 success\n");
43         exit(EXIT_FAILURE);
44     }

```

In this step, the "victim" socket for the exploit is created. `s` is reinitialized with no port, no peer, no bind, and no transport.

We want the `connect` to fail in a specific way. With this connection attempt, the kernel prepares the socket, selects the transport and attempts to assign it a local port. The 24 ports we set up earlier are used to steer the process along this error path so that the transport is set but the socket remains in the unbound list.

```

47     addr.svm_cid = VMADDR_CID_NONEXISTING;
48     if (!connect(s, (struct sockaddr *)&addr, alen)) {
49         fprintf(stderr, "Unexpected connect() #2 success\n");
50         exit(EXIT_FAILURE);
51     }
52     /* connect() #2 failed: transport unset, sk ref dropped? */

```

The CID is then set to a non-existent type to force the next connection attempt to fail; this one proceeds just like the previous one, and we again expect it to fail, but for a different reason. This causes the transport to be unset and, at the same time, the reference count to be decreased even if the object is still existing.

```

54     addr.svm_cid = VMADDR_CID_LOCAL;
55     addr.svm_port = VMADDR_PORT_ANY;
56

```

```

57      /* Vulnerable system may crash now. */
58      bind(s, (struct sockaddr *)&addr, alen);
59
60      close(s);
61      while (i--)
62          close(sockets[i]);
63
64      control_writeln("DONE");

```

The final step demonstrates how the vulnerability was triggered, showing that, on a vulnerable system, attempting to bind a socket to an invalid object causes a crash.

3.4 Cause and functions involved

In order to understand exactly why the bug occurs, we can start by looking at the patch the developers applied to fix it. Before the patch [41] (kernel versions until 6.6.78 [42]), the vulnerable code in `af_vsock` looked like this:

```

1      void vsock_remove_sock(struct vsock_sock *vsk)
2      {
3          vsock_remove_bound(vsk);
4          vsock_remove_connected(vsk);
5      }

```

while in the version after the patch (6.6.79) it looked like this:

```

1      void vsock_remove_sock(struct vsock_sock *vsk)
2      {
3          /* Transport reassignment must not remove the binding. */
4          if (sock_flag(sk_vsock(vsk), SOCK_DEAD))
5              vsock_remove_bound(vsk);
6
7          vsock_remove_connected(vsk);
8      }

```

The maintainers added a simple `if` that checks if the socket is really "dead" before removing the binding. We can better understand how it works by working backward, starting with the call to the `vsock_remove_bound` function.

Before examining the function itself, we need to explain how the **bound tables** approach works. There are two global (as all sockets on the system are placed within them) tables, one for **bound sockets** (sockets that have specified an address that they are responsible for) and one for **connected sockets** (sockets that have established a connection with another socket). The bound table contains an extra entry for a list of **unbound sockets**. These tables are managed as *intrusive lists* and are contained directly within the socket structs.

In the test case, the newly created `SOCK_SEQPACKET` socket is initially inserted into the unbound list, since no local address has been assigned to it yet. The

first `connect()` attempt implicitly tries to bind the socket, but the previously allocated consecutive ports force this operation to fail. As a result, the socket remains in the unbound list, although a transport has already been assigned to it.

The second `connect()` changes the destination so that a different transport selection is required. During this reassignment, the previous transport is released and the socket is removed through the normal bound-socket removal path, but the socket was never actually moved from the unbound list to one of the regular bound-table buckets. The vulnerable code therefore treats an unbound socket as if it were normally bound and drops a reference that should have been preserved. The subsequent `bind()` then operates on a socket whose reference count may already be incorrect, exposing the use-after-free condition.

Now that we've covered how bound tables work, we can examine the contents of `vsock_remove_bound`. Since the original function calls a wrapper that contains mutual exclusion instructions and the entire function that contains the actual logic, for simplicity we'll focus directly on the latter:

```

1      static void __vsock_remove_bound(struct vsock_sock *vsk)
2      {
3          list_del_init(&vsk->bound_table);
4          sock_put(&vsk->sk);
5      }

```

The `list_del_init` statement removes the socket from the bound table, while the `sock_put` function contains the statement responsible for decrementing the reference counter for the current socket and, if necessary, destroying it if the decremented reference was the last one:

```

1      static inline void sock_put(struct sock *sk)
2      {
3          if (refcount_dec_and_test(&sk->sk_refcnt))
4              sk_free(sk);
5      }

```

This means that, if the socket was not actually unbound, its reference counter was still decremented and the memory was freed, even though a valid reference to it still existed.

Chapter 4

Setup and tests with UAFX

The first test we conducted was the one with UAFX. Although the paper [23] explicitly states that UAFs caused by refcounts are not supported, we decided to test the tool as-is to see if it would still be able to detect the vulnerability, and then to modify it to remove the limitations imposed by the researchers and observe any changes in the results.

4.1 Setup

We started by downloading the kernel source code for version 6.6.75 from the official repository, a vulnerable version that is suitable for CVE-2025-21756 and officially supported by UAFX. To set up the environment, we installed the necessary dependencies (the most important of which include Python and LLVM version 14), `gclang`, a wrapper for the Clang compiler used to compile the source code of the relevant modules in C and generate LLVM bitcode files, and `gllvm`, a Go porting of LLVM which relies on it to extract the bitcode from the kernel source files.

UAFX also requires `z3`, an SMT solver (a type of tool that checks whether a logical formula is satisfiable based on mathematical rules). It acts as a logic engine and, in our case, is used to check whether a certain path is feasible during static analysis.

Before moving on, let's introduce some terms used in the explanation:

- **bitcode**: represents the target code in LLVM Intermediate Representation (IR), the level at which UAFX performs its static analysis passes. This intermediate form is essential because it allows the tool to accurately analyze pointers and perform escape-fetch analysis, which require a code structure with more semantic information than the final binary to map the relationships between pointers even when they span multiple kernel entry functions.
- **object files (.o)**: the result of partially compiling the source code into machine code. The .o files are generated alongside the .bc files to ensure that the build environment is consistent. In our case, while the security analysis is performed on the bitcode, the original object files serve to confirm

that the code is "compilable" and ready to be converted into a version that the tool can analyze.

After the initial steps, we considered two possible alternatives:

1. compiling with `gllvm/wllvm`: these wrappers produce standard binaries and retain a reference to the bitcode, from which a single `.bc` file is then extracted;
2. building the kernel with pure Clang/LLVM + a custom pipeline for merging `.o.bc` files.

Since UAFX requires a single `.bc` file by default, we decided to go with the first option. `gllvm` and `wllvm` act as compiler wrappers; they generate the `.bc` files for the objects, and then `extract-bc` or `get-bc` link them into a single module, on which we can run the tool directly.

We configured some bash environment variables to ensure that LLVM and clang use version 14:

```

1  export PATH=/usr/lib/llvm-14/bin:$PATH:$(go env GOPATH)/bin
2  export LLVM_COMPILER=clang
3  export LLVM_COMPILER_PATH=/usr/lib/llvm-14/bin
4  export LLVM_CC_NAME=clang-14
5  export LLVM_CXX_NAME=clang++-14
6  export LLVM_LINK_NAME=llvm-link
7  export LLVM_AR_NAME=llvm-ar-14

```

We also needed to setup the kernel source code in order to build it and extract the bitcode:

```

1  cd ~/src/linux-6.6.75
2  make mrproper
3  make LLVM--14 defconfig
4  make LLVM--14 menuconfig

```

These commands allow us to define which modules we're going to launch UAFX on, making the compiled kernel a valid target. Specifically:

- `make mrproper` performs a complete cleanup of the kernel source tree, removing not only the object files and binaries generated by previous builds (as `make clean` would do), but also the `.config` configuration file and all automatically generated files, such as the header files produced during the build process. This ensured that we started with a completely clean source tree, eliminating any residual files that could affect the subsequent compilation. This has been particularly useful for us when running multiple consecutive tests with (slightly) different configurations, so that we can always start from a neutral baseline.

- `make LLVM=-14 defconfig` generates a default kernel configuration for the current architecture, using Clang/LLVM version 14 as the compilation toolchain instead of GCC. The configuration generated by `defconfig` includes a minimal set of default options, suitable for most test environments, and served as a starting point for our manual adjustments.
- `make LLVM=-14 menuconfig` opens an interactive menu-based interface that allows us to modify the configuration generated in the previous step. In our context, it was used to explicitly enable all components of the vsock subsystem (`VSOCK`, `VSOCK_LOOPBACK`, `VSOCK_DIAG`, `VMW_VSOCK_VIRTIO_TRANSPORT`, and `VMW_VSOCK_VIRTIO_TRANSPORT_COMMON`), so that the compiled kernel was a suitable target for the analysis performed with UAFX.

Next, we have the final configuration steps and the launch of the actual build:

```
1 make LLVM=-14 olddefconfig
2 make -j "$(nproc)" LLVM=-14 prepare modules_prepare scripts
```

With `make LLVM=-14 olddefconfig`, we reset the `.config` file to the default options for all configuration entries that were not explicitly set via `menuconfig`, because after any manual changes to the configuration, there may be unresolved symbols or unset values that could cause problems during compilation.

The following `make` command performs a series of preparatory steps that are essential for compiling external modules (in our case, UAFX). Here is a detailed explanation of the parameters:

- `prepare` generates the headers and internal kernel data structures that modules need to know about at compile time, including `autoconf.h` and the symbol tables;
- `modules_prepare` extends the preparation process to include everything needed to compile loadable modules (`.ko`), specifically the dependency files and version data for the symbols exported by the kernel (`Module.symvers`);
- `scripts` compiles the auxiliary build tools located in `scripts/`, such as `recordmcount` and the various helper programs used by the kernel build system;
- `-j "$(nproc)"` parallelizes the work across all available logical cores, speeding up the compilation steps.

Once the environment is set up, we start with the actual compilation:


```

1  make -j"$(nproc)" LLVM=-14 \
2    CC=gclang HOSTCC=clang-14 HOSTCXX=clang++-14 \
3    LD=ld.lld-14 AR=llvm-ar-14 NM=llvm-nm-14 \
4    OBJCOPY=llvm-objcopy-14 \
5    OBJDUMP=llvm-objdump-14 \
6    STRIP=llvm-strip-14 \
7    net/vmw_vsock/vsock.o \
8    net/vmw_vsock/vmw_vsock_virtio_transport.o \
9    net/vmw_vsock/vmw_vsock_virtio_transport_common.o \
10   net/vmw_vsock/vsock_loopback.o \
11   net/vmw_vsock/vsock_diag.o \
12   net/socket.o

```

This command initiates selective compilation of only the object modules required for the analysis, explicitly specifying the entire LLVM/Clang toolchain to be used. Just as we did with the previous one, let's take a look at its structure:

- `CC=gclang` replaces the C compiler with `gclang`, which, as we mentioned earlier, not only produces the standard object files but also generates the corresponding LLVM bitcode in parallel, which will be used by UAFX;
- `HOSTCC=clang-14` and `HOSTCXX=clang++-14` specify the compiler to use for build tools run on the host (such as the tools in `scripts/`), which is different from the target compiler;
- `LD`, `AR`, `NM`, `OBJCOPY`, `OBJDUMP`, and `STRIP` replace the corresponding `binutils` tools with their LLVM counterparts.

After the `STRIP` parameter, we have a set of targets that correspond to the `vsock` subsystem modules selected during the `menuconfig` phase, plus `net/socket.o`, which implements the generic socket interface at the kernel level and is required to resolve symbol dependencies. The result is a set of `.o` files, from which we extract the bitcode:

```

1  get-bc -o
2    ../uafx/_bc/vsock.bc
3    net/vmw_vsock/vsock.o
4  get-bc -o
5    ../uafx/_bc/vmw_vsock_virtio_transport.bc
6    net/vmw_vsock/vmw_vsock_virtio_transport.o
7  get-bc -o
8    ../uafx/_bc/vmw_vsock_virtio_transport_common.bc
9    ↪ net/vmw_vsock/vmw_vsock_virtio_transport_common.o
10 get-bc -o
11   ../uafx/_bc/vsock_loopback.bc
12   net/vmw_vsock/vsock_loopback.o
13 get-bc -o
14   ../uafx/_bc/vsock_diag.bc
15   net/vmw_vsock/vsock_diag.o

```

```

15 get-bc -o
16     ../uafx/_bc/socket.bc
17     net/socket.o

```

Here we use `get-bc`, a utility from the `wllvm` toolkit (on which `gllvm` is based), which retrieves the LLVM bitcode previously embedded by `gclang` within each `.o` object file and exports it to standalone `.bc` files. As mentioned, LLVM bitcode is an intermediate representation of the source code, independent of the target architecture, that preserves high-level semantic information that would otherwise be lost in traditional compilation to machine code. The elements of interest to us, for use with UAFX, are the types, the call graph, and the relationships between instructions (which are extremely important for escape-fetch-based cross-entry alias analysis, specifically in the reconstruction of object paths). The `socket.bc` file is included to allow UAFX to resolve the symbols in the kernel's generic socket layer that the `vsock` modules reference.

As the final step in the general configuration, we set up the list of functions to be considered as entry points by UAFX, a partial list of which is provided below:

```

1 vsock_create
2 vsock_bind
3 vsock_remove_sock
4 ...
5 __sys_socket
6 __sys_bind
7 __sys_connect
8 ...

```

It is important to note that some of the functions we have included are defined within the Linux kernel as syscalls (TODO - breve spiegazione syscalls, eventuale funzionamento definizione in kernel linux ecc.). This presented another potential obstacle because the UAFX paper does not mention any support for functions of this type, nor does it explicitly rule them out. Since the objects we are considering necessarily go through these functions as well, we had to add them to the set of entry functions for UAFX.

At this point, we had two options:

1. include them directly as separate modules (even though the paths between modules would remain incomplete);
2. link the modules and run UAFX on a cumulative bitcode.

The first option involved running UAFX on the modules separately. In this case, only UAFs related to issues within individual modules would have been detected (and thus much less likely to have gone undetected than a UAF affecting multiple modules). With this approach, the configuration ends here.

The second method, on the other hand, requires a few additional configuration steps. As mentioned earlier, the goal was to combine all the modules in question into a single cumulative module so that it could be fed to UAFX all at once, taking into account all the functions of all the modules.

As a first step in this process, we had to disassemble some of the modules so that we could bulk-rename certain references that would have been repeated in the other modules. This was necessary because

4.2 Test execution

Chapter 5

Setup and tests with CountDown

Chapter 6

Conclusions

Bibliography

- [1] The GNU Allocator https://sourceware.org/glibc/manual/2.27/html_node/The-GNU-Allocator.html
- [2] Efficiency Considerations for malloc https://sourceware.org/glibc/manual/2.23/html_node/Efficiency-and-Malloc.html
- [3] Efficient detection of dangling pointer error for C/C++ programs https://www.researchgate.net/publication/319596347_Efficient_detection_of_dangling_pointer_error_for_CC_programs, 2017
- [4] Preventing privilege escalation https://www.academia.edu/1960780/Preventing_privilege_escalation, 2003
- [5] A method for overlapping and erasure of lists <https://dl.acm.org/doi/10.1145/367487.367501>, 1960
- [6] Smashing The Stack For Fun And Profit https://www.andrew.cmu.edu/course/18-330/2025f/reading/One_1996_Smashing%20The%20Stack%20For%20Fun%20And%20Profit.pdf, 1996
- [7] Detecting deadlock, double-free and other abuses in a million lines of linux kernel source https://www.academia.edu/1413564/Detecting_deadlock_double_free_and_other_abuses_in_a_million_lines_of_linux_kernel_source, 2006
- [8] CWE-416: Use After Free <https://cwe.mitre.org/data/definitions/416.html>, 2006
- [9] The quantum state of Linux kernel garbage collection CVE-2021-0920 (Part I) <https://projectzero.google/2022/08/the-quantum-state-of-linux-kernel.html>, 2022
- [10] Linux Kernel Exploitation - CVE-2025-21756: Attack of the Vsock <https://hoefler.dev/articles/vsock.html>, 2025
- [11] Heap Feng Shui in Javascript <https://blackhat.com/presentations/bh-usa-07/Sotirov/Whitepaper/bh-usa-07-sotirov-WP.pdf>, 2007
- [12] On the Effectiveness of Address-Space Randomization https://www.andrew.cmu.edu/course/18-330/2025f/reading/Shacham_2004_ASLR_Effectiveness.pdf, 2004

- [13] Control-flow integrity principles, implementations, and applications <https://dl.acm.org/doi/10.1145/1609956.1609960>, 2009
- [14] Linux Market Share Statistics 2026: Desktop, Server and Enterprise Adoption Trends <https://fosspost.org/linux-market-share-statistics/>, 2026.
- [15] Kernel Object in Microsoft Windows Docs <https://learn.microsoft.com/en-us/windows/win32/sysinfo/kernel-objects>
- [16] sched.h in Linux Kernel <https://elixir.bootlin.com/linux/v7.1.8/source/include/linux/sched.h>
- [17] struct socket in Linux Kernel <https://elixir.bootlin.com/linux/v7.1.8/source/include/linux/net.h#L127>
- [18] struct sock in Linux Kernel <https://elixir.bootlin.com/linux/v7.1.8/source/include/net/sock.h#L244>
- [19] struct inode in Linux Kernel <https://elixir.bootlin.com/linux/v7.1.8/source/include/linux/fs.h#L767>
- [20] struct sk_buff in Linux Kernel <https://elixir.bootlin.com/linux/v7.1.8/source/include/linux/skbuff.h#L775>
- [21] The Slab Allocator: An Object-Caching Kernel Memory Allocator https://people.eecs.berkeley.edu/~kubitron/courses/cs194-24-S14/hand-outs/bonwick_slab.pdf, 1994
- [22] The SLUB allocator <https://lwn.net/Articles/229984/>, 2007
- [23] Statically Discover Cross-Entry Use-After-Free Vulnerabilities in the Linux Kernel <https://www.ndss-symposium.org/wp-content/uploads/2025-559-paper.pdf>, 2025
- [24] LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation <https://dl.acm.org/doi/10.5555/977395.977673>, 2004
- [25] Clang Static Analyzer <https://github.com/llvm/llvm-project/blob/main/clang/lib/StaticAnalyzer/README.txt>, 2008
- [26] Countdown: Refcount-guided Fuzzing for Exposing Temporal Memory Errors in Linux Kernel <https://dl.acm.org/doi/epdf/10.1145/3658644.3690320>, 2024
- [27] Syzkaller: unsupervised coverage-guided kernel fuzzer <https://github.com/google/syzkaller/blob/master/docs/internals.md>, 2015
- [28] Infer: An Automatic Program Verifier for Memory Safety of C Programs <https://scispace.com/pdf/infer-an-automatic-program-verifier-for-memory-safety-of-c-1d048xu12f.pdf>, 2011

- [29] GNU Compiler Collection (GCC) <https://github.com/gcc-mirror/gcc>, 1987
- [30] Spatio-Temporal Context Reduction: A Pointer-Analysis-Based Static Approach for Detecting Use-After-Free Vulnerabilities <https://dl.acm.org/doi/epdf/10.1145/3180155.3180178>, 2018
- [31] Cppcheck: static analysis of C/C++ code <https://github.com/cppcheck/cppcheck>, 2007
- [32] SVF: Static Value-Flow Analysis Framework for Source Code <https://github.com/SVF-tools/SVF>, 2016
- [33] FreeWill: Automatically Diagnosing Use-after-free Bugs via Reference Miscounting Detection on Binaries <https://www.usenix.org/system/files/sec22-he-liang.pdf>, 2022
- [34] LinKRID: Vetting Imbalance Reference Counting in Linux kernel with Symbolic Execution <https://www.usenix.org/system/files/sec22-liu-jian.pdf>, 2022
- [35] CRCCount: Pointer Invalidation with Reference Counting to Mitigate Use-after-free in Legacy C/C++ https://www.ndss-symposium.org/wp-content/uploads/2019/02/ndss2019_05A-4_Shin_paper.pdf, 2019
- [36] UAFChecker: Scalable Static Detection of Use-After-Free Vulnerabilities <https://dl.acm.org/doi/epdf/10.1145/2660267.2662394>, 2014
- [37] Effective Static Analysis of Concurrency Use-After-Free Bugs in Linux Device Drivers <https://www.usenix.org/system/files/atc19-bai.pdf>, 2019
- [38] SinkFinder: Harvesting Hundreds of Unknown Interesting Function Pairs with Just One Seed <https://dl.acm.org/doi/epdf/10.1145/3368089.3409678>, 2020
- [39] Goshawk: Hunting Memory Corruptions via Structure-Aware and Object-Centric Memory Operation Synopsis <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9833613>, 2022
- [40] Detecting Kernel Refcount Bugs with Two-Dimensional Consistency Checking <https://yuanxizhang.github.io/paper/cid-security21.pdf>, 2021
- [41] vsock: Keep the binding until socket destruction, Linux kernel patch <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=fcdd2242c0231032fc84e1404315c245ae56322a>, 2025
- [42] af_vsock module, Linux kernel code, v6.6.78 https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/net/vmw_vsock/af_vsock.c?h=v6.6.78, 2025

- [43] Z3: an efficient SMT solver <https://dl.acm.org/doi/10.5555/1792734.1792766>, 2008